



Act as a senior software engineer and DSA expert preparing me for top product-based companies (FAANG-level).

I will provide a YouTube video summary or transcript. The summary may contain MULTIPLE TOPICS.

Your job is to:

1. First identify ALL topics
2. Then teach ONLY ONE topic at a time in depth
3. Complete the full explanation of that topic before moving to the next

=====

STEP 1: Topic Identification

- Extract all main topics from the summary
- List them clearly
- Do NOT explain yet

=====

STEP 2: Start Teaching (One Topic at a Time)

For EACH topic, follow this structure strictly:

#####

TOPIC: <Topic Name>

#####

1. Simple & Detailed Theory (Teacher Style)

- Explain in very simple words (like teaching a beginner)
- Use real-life analogy
- Build intuition step-by-step
- Explain WHY this concept is used

2. Core Concepts (Deep Understanding)

- Explain all important details
- Internal logic and working
- Do not skip small details

3. Examples (Very Important)

- Give 2–3 examples
- Explain each step clearly

4. Important Interview Notes $\triangle$

- Key points frequently asked in interviews
- Tricky observations
- Common confusions

5. Problem Identification

- Extract all problems from the summary related to this topic
- If missing, add standard interview problems

6. Solve Each Problem (VERY IMPORTANT)

For EACH problem:

☒ Problem Statement

- Explain clearly (input, output, constraints)

☒ Intuition (How to Think)

- Step-by-step thinking process
- How to approach in interview

☒ Pattern Used

- Identify pattern (Two Pointer, Sliding Window, etc.)

☒ Brute Force Approach

- Explanation + code
- Why inefficient

☒ Optimized Approach

- Step-by-step improvement
- Key idea

☒ Code (Java Preferred)

- Clean and readable
- With comments

☒ Detailed Dry Run ★

- Step-by-step execution
- Show variable changes
- Explain WHY each step happens

☒ Edge Cases & Mistakes

- Important corner cases
- Common mistakes

7. Topic Summary (Quick Revision)

- Key points
- Patterns
- Important tricks

STEP 3: Move to Next Topic

- Only after completing current topic fully
- Repeat same structure for next topic

Important Instructions:

- Teach like a human teacher (not robotic notes)
- Focus heavily on intuition + code + dry run
- Do NOT skip steps
- Do NOT mix topics
- Depth is more important than speed
- Assume preparation for high-package companies

Please provide the YouTube video summary or transcript you'd like me to analyze. Once I have it, I'll identify all topics clearly and start teaching them one by one in depth, following the exact structure you outlined. [\[1\]](#) [\[2\]](#)



STEP 1: Topic Identification

Here are all the main topics extracted from the video summary:

1. Prime Numbers
2. Sieve of Eratosthenes
3. Square Roots
4. Factors
5. Modulo Properties
6. HCF/GCD & LCM

I'll now teach these topics **ONE AT A TIME** in depth, starting with the first one. Each topic will be fully completed before moving to the next.

#####

TOPIC: Prime Numbers

#####

1. Simple & Detailed Theory (Teacher Style)

Imagine you're at a fruit market and you want to buy only "special" oranges that can't be divided into smaller groups of oranges - you can only buy them as single whole oranges or the entire basket. These special oranges represent **prime numbers**.

A prime number is a natural number **greater than 1** that has **no positive divisors other than 1 and itself**. Think of it like a "prime real estate" - it doesn't share its space with anyone else!

Why do we care about primes in DSA interviews?

- Primes are the "building blocks" of all numbers (just like atoms are building blocks of matter)
- Many problems involve counting primes, checking primality, or using prime properties
- They're everywhere in cryptography, competitive programming, and optimization problems

Step-by-step intuition:

1. Start with number 2 ✓ (only divisible by 1 and 2)
2. Check 3 ✓ (only divisible by 1 and 3)
3. Check 4 ✗ (divisible by 1, 2, 4)

4. Check 5 ✓ (only divisible by 1 and 5)

5. And so on...

Key insight: To check if N is prime, you only need to check divisors from 2 to $\sqrt{N}$ (square root of N), because if N has a factor $> \sqrt{N}$, it must also have a factor $< \sqrt{N}$.

2. Core Concepts (Deep Understanding)

Formal Definition: A number N is prime if for all integers $i (2 \leq i \leq \sqrt{N})$, $N \% i \neq 0$.

Mathematical Properties:

- There are infinitely many primes (Euclid's theorem)
- Prime gaps increase as numbers get larger
- Goldbach conjecture: Every even integer > 2 is sum of two primes (unproven!)

Naive Primality Test Algorithm:

```
function isPrime(N):
    if N ≤ 1: return false
    if N ≤ 3: return true
    if N % 2 == 0 or N % 3 == 0: return false

    i = 5
    while i * i ≤ N:
        if N % i == 0 or N % (i + 2) == 0:
            return false
        i += 6
    return true
```

Why check up to $\sqrt{N}$ only?

If $N = 100$, $\sqrt{N} = 10$
Factors come in pairs: (1,100), (2,50), (4,25), (5,20), (10,10)
If we find factor > 10 , its pair must be < 10 !
So checking 2 to 10 covers ALL possible factors.

Time Complexities:

- Naive: $O(N) \rightarrow$ Too slow!
- Better: $O(\sqrt{N}) \rightarrow$ Interview standard
- Optimized with $6k \pm 1$: $O(\sqrt{N}/3) \rightarrow$ Production ready

3. Examples (Very Important)

Example 1: $N = 17$

Check divisors: 2,3,4 (since $\sqrt{17} \approx 4.1$)

$17\%2=1$, $17\%3=2$, $17\%4=1 \rightarrow$ PRIME!

Example 2: $N = 39$

Check divisors: 2,3,4,5,6 (since $\sqrt{39} \approx 6.2$)
 $39\%3=0 \rightarrow$ NOT PRIME!

Example 3: $N = 97$

Check up to $\sqrt{97} \approx 9.8 \rightarrow 2,3,5,7$
 $97\%2=1$, $97\%3=1$, $97\%5=2$, $97\%7=6 \rightarrow$ PRIME!

4. Important Interview Notes

Frequently Asked:

1. "Write isPrime(N) function" $\rightarrow$ Always mention $O(\sqrt{N})$ optimization!
2. "Count primes till N" $\rightarrow$ Hint towards Sieve (next topic)
3. "Check if number is prime using recursion"
4. "Find next prime after N"

Tricky Observations:

- 1 is NOT prime! (common mistake)
- 2 is the only even prime! (handle separately)
- Negative numbers & 0 are NOT prime
- For $N=1$, directly return false

Common Confusions:

- ✗ Wrong: Check all divisors till N
- ✓ Right: Check only till $\sqrt{N}$
- ✗ Wrong: For $N=9$, check 2,3 $\rightarrow$ return true (missed $3*3$)
- ✓ Right: $i*i \leq N$ catches perfect squares

5. Problem Identification

From video summary: Primality Test - Check if given number N is prime, optimized by checking divisors up to $\sqrt{N}$.

Additional standard problems:

1. Primality Test (main problem)
2. Count Primes Till N
3. Next Prime Number

6. Solve Each Problem (VERY IMPORTANT)

📄 Problem Statement

Primality Test

- Input: Integer N ($2 \leq N \leq 10^{12}$)
- Output: "Yes" if prime, "No" otherwise
- Constraints: Single test case, large N possible

📄 Intuition (How to Think)

1. Handle base cases: $N < 2 \rightarrow \text{false}$, $N = 2 \rightarrow \text{true}$, $N \text{ even} \rightarrow \text{false}$
2. Check divisibility by 3
3. Use $6k \pm 1$ optimization: Only check numbers of form $6k-1$ or $6k+1$
4. Stop at $\sqrt{N}$ - no need to check further
5. Interview tip: Always draw $\sqrt{N}$ insight on whiteboard!

📄 Pattern Used

Math + Early Termination + Optimized Loop

📄 Brute Force Approach

```
// O(N) - Too slow for N=10^12
public boolean isPrimeBrute(int n) {
    if (n <= 1) return false;
    for (int i = 2; i < n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
```

Why inefficient? For $N=10^{12}$, loops 1 trillion times!

📄 Optimized Approach

1. Handle 2,3 separately (all primes >3 are $6k \pm 1$ form)
2. Check i and $i+2$ in single iteration
3. Use $i*i \leq n$ instead of $\text{Math.sqrt}()$ for precision
4. Time: $O(\sqrt{N}/3)$ instead of $O(\sqrt{N})$

☒ Code (Java Preferred)

```
public class PrimeChecker {
    public static boolean isPrime(long n) {
        // Base cases
        if (n <= 1) return false;
        if (n <= 3) return true;
        if (n % 2 == 0 || n % 3 == 0) return false;

        // Check 6k ± 1 up to sqrt(n)
        for (long i = 5; i * i <= n; i += 6) {
            if (n % i == 0 || n % (i + 2) == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        long n = 97;
        System.out.println(isPrime(n) ? "Yes" : "No"); // Yes
    }
}
```

☒ Detailed Dry Run ☆

Input: N = 97

```
Step 1: n=97>3 ✓
Step 2: 97%2=1, 97%3=1 ✓
Step 3: i=5 (5*5=25 <=97)
        97%5=2, 97%7=6 ✓
Step 4: i=11 (11*11=121 >97) STOP!
Result: PRIME ✓
```

Visual:
i=5: 5×5=25 ≤97 → check 97%5, 97%7
i=11: 11×11=121>97 → EXIT LOOP

Input: N = 91

```
i=5: 91%7=0 → IMMEDIATELY return false!
Super fast detection!
```

☒ Edge Cases & Mistakes

Edge Cases:

1. N=1 → false (not prime)
2. N=2 → true (only even prime)
3. N=4 → false (smallest composite)

4. $N=10^{12}+7 \rightarrow \text{true}$ (large prime)

5. $N=1_000_000_007 \rightarrow \text{true}$

Common Mistakes:

1. ✗ Using `Math.sqrt(n)` $\rightarrow$ floating precision issues
✓ Use `i*i <= n`

2. ✗ Forgetting $n \leq 1$ case

3. ✗ Missing perfect squares ($9=3*3$)

4. ✗ Using `int` for $n=10^{12} \rightarrow$ Use `long`!

7. Topic Summary (Quick Revision)

Key Points:

- Prime = Natural number >1 with no divisors except 1 and itself
- Check only till $\sqrt{N}$ - interview must-know!
- $6k \pm 1$ optimization - shows deeper understanding
- Handle 2,3 separately for best performance

Important Patterns:

1. Early termination (even numbers)
2. $\sqrt{N}$ optimization (factor pairs)
3. Wheel factorization ($6k \pm 1$)
4. `i*i <= n` (avoid floating point)

Tricks for Interviews:

1. Always mention time complexity: $O(\sqrt{N}) \rightarrow O(\sqrt{N}/3)$
2. Draw $\sqrt{N}$ insight: "Factors come in pairs!"
3. Code both brute + optimized
4. Test edge cases: 1,2,4, prime, composite

Topic 1 COMPLETE! Ready for next topic? Let me know or I'll continue with Sieve of Eratosthenes in the next response.

ok stop i will provide you different summary

Understood! I'll stop the previous topic sequence and wait for your new YouTube video summary or transcript.

Just paste it when you're ready, and I'll:

1. Identify ALL topics clearly
2. Start teaching ONE topic at a time in complete depth
3. Follow the exact structure you specified

Ready when you are! ☒

STEP 1: Topic Identification

Here are all the main topics extracted from the video summary:

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List

Note: Introduction/Basics is foundational theory (covered in Singly topic).

Operations/problems are implementation-specific to each type. I'll teach ONE topic at a time, extracting relevant problems and adding standard interview ones.

Starting with the first topic now!

#####

TOPIC: Singly Linked List

#####

1. Simple & Detailed Theory (Teacher Style)

Picture a **train** made of individual cars (nodes). Each car carries **passengers/data** and has a **hook/pointer** to the next car. The **engine** is the **head**, and the last car has no hook (null).

Unlike an array (fixed parking lot where cars must be side-by-side), linked lists let cars be scattered anywhere - you just follow the hooks!

Why use Singly Linked Lists?

- Dynamic size (arrays need resizing)
- Efficient insertions/deletions at head ($O(1)$ vs array's $O(N)$)
- Memory efficient for sparse data
- Perfect for queues, stacks, browser history

Step-by-step intuition:

1. Create Node: data + next
2. Head points to first node
3. Tail (optional) points to last node for $O(1)$ end ops
4. Traverse: Start at head, follow next until null
5. Insert: Adjust pointers (like attaching train cars)
6. Delete: Skip node by linking prev $\rightarrow$ next

Real-life: Browser "Back" button - each page links to previous!

2. Core Concepts (Deep Understanding)

Node Structure:

```
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

Key Components:

- **Head:** Entry point (null = empty list)
- **Tail:** Optional, speeds up end operations
- **Size:** Track length (helps index ops)
- **Temporary pointer (temp/newNode/prev):** Never modify head directly!

Internal Logic:

Insertion at index i :

1. If $i=0$: `newNode.next = head; head = newNode`
2. Else: Traverse to $i-1$, then `(prev.next = newNode; newNode.next = oldNode)`

Deletion at index i :

1. If $i=0$: `head = head.next`
2. Else: Traverse to $i-1$, `prev.next = prev.next.next`

Time Complexities:

Operation	With Tail	Without Tail
Insert Head	$O(1)$	$O(1)$
Insert Tail	$O(1)$	$O(N)$
Insert Index	$O(N)$	$O(N)$
Delete Head	$O(1)$	$O(1)$
Delete Tail	$O(1)$	$O(N)$
Search	$O(N)$	$O(N)$

Memory: $O(N)$ space, non-contiguous allocation.

3. Examples (Very Important)

Example 1: Insert at Head

```
Initial: 10→20→30 (head=10)
Insert 5 at head: 5→10→20→30 (head=5)
Steps: new.next=oldHead; head=new;
```

Example 2: Delete from End

```
List: 10→20→30→null
Traverse: temp=10→20→30 (prev=20 when temp.next==null)
prev.next=null → 10→20→null
```

Example 3: Insert at Index 2

```
List: 10→20→30
Insert 25 at index 2: temp goes 10(0)→20(1), prev=20
20.next=25; 25.next=30 → 10→20→25→30
```

4. Important Interview Notes ⚠

Frequently Asked:

1. "Implement insert/delete at all positions"
2. "Reverse a linked list" (follow-up)
3. "Detect cycle" (Floyd's tortoise-hare)
4. "Find middle node" (slow-fast pointers)
5. "Merge two sorted lists"

Tricky Observations:

- Null head handling (empty list)
- Single node cases
- Index out of bounds (index >= size)
- Tail helps but complicates code - interviews prefer simple head-only

Common Confusions:

- ✗ Modify head directly → lose list!
✓ Use temp = head;
- ✗ Forget to update prev.next in delete
✓ Always: prev.next = prev.next.next
- ✗ Infinite loop in display → no null check
✓ while(temp != null)

Pro Tip: Always draw pointers before coding! Interviewers love this.

5. Problem Identification

From video summary (Singly Linked List Operations):

1. Insertion at first position
2. Insertion at last position
3. Insertion at specific index
4. Deletion from first position
5. Deletion from last position
6. Deletion at specific index/value
7. Display/Traversing the list

Additional standard interview problems:

8. Reverse the list
9. Find length

6. Solve Each Problem (VERY IMPORTANT)

☒ Problem 1: Insertion at First Position

Problem Statement

- Input: Head of list, value to insert
- Output: Updated head
- Constraints: $0 \leq N \leq 10^5$

Intuition: Like adding a new engine car in front. Just point new node to old head, update head.

Pattern Used: Pointer Manipulation

Brute Force: N/A (already $O(1)$)

Optimized Approach: Direct pointer update.

Code (Java):

```
public Node insertFirst(Node head, int data) {
    Node newNode = new Node(data);
    newNode.next = head; // New points to old head
    head = newNode;      // Head now new node
    return head;
}
```

Dry Run: List: 10 → 20

```
newNode(5).next = 10  
head = 5 → 5→10→20 ✓
```

Edge Cases: Empty list (head=null → new list), single node.

📌 Problem 2: Insertion at Last Position (No Tail)

Problem Statement

- Input: Head, data
- Output: Updated head

Intuition: Traverse to end (null), attach new node.

Code:

```
public Node insertLast(Node head, int data) {  
    Node newNode = new Node(data);  
    if (head == null) return newNode;  
  
    Node temp = head;  
    while (temp.next != null) {  
        temp = temp.next;  
    }  
    temp.next = newNode;  
    return head;  
}
```

Dry Run: 10→20

```
temp=10→20 (20.next=null)  
20.next=30 → 10→20→30
```

📌 Problem 3: Insertion at Specific Index

Problem Statement

- Input: Head, index (0-based), data
- Output: Updated head

Intuition: Traverse to index-1, insert between prev and target.

Code:

```
public Node insertAtIndex(Node head, int index, int data) {  
    Node newNode = new Node(data);  
    if (index == 0) {  
        newNode.next = head;  
        return newNode;  
    }
```

```

Node temp = head;
for (int i = 0; i < index - 1 && temp != null; i++) {
    temp = temp.next;
}
if (temp != null) {
    newNode.next = temp.next;
    temp.next = newNode;
}
return head;
}

```

Dry Run: Index=1, data=15 in 10→20→30

```

i=0: temp=10
newNode.next=20
10.next=15 → 10→15→20→30

```

☒ Problem 4: Deletion from First Position

Code:

```

public Node deleteFirst(Node head) {
    if (head == null) return null;
    return head.next;
}

```

Dry Run: 10→20 → head=20

☒ Problem 5: Deletion from Last Position

Intuition: Traverse to second last, set its next=null.

Code:

```

public Node deleteLast(Node head) {
    if (head == null || head.next == null) return null;

    Node temp = head;
    while (temp.next.next != null) {
        temp = temp.next;
    }
    temp.next = null;
    return head;
}

```

Dry Run: 10→20→30

```

temp=10→20 (20.next.next=null)

```

20.next=null → 10→20

☒ Problem 6: Deletion at Specific Index

Code:

```
public Node deleteAtIndex(Node head, int index) {
    if (head == null) return null;
    if (index == 0) return head.next;

    Node temp = head;
    for (int i = 0; i < index - 1 && temp != null; i++) {
        temp = temp.next;
    }
    if (temp != null && temp.next != null) {
        temp.next = temp.next.next;
    }
    return head;
}
```

☒ Problem 7: Display/Traverse

Code:

```
public void display(Node head) {
    Node temp = head; // Don't modify head!
    while (temp != null) {
        System.out.print(temp.data + " → ");
        temp = temp.next;
    }
    System.out.println("null");
}
```

Why temp? Protects original head pointer.

Additional Problems (Brief):

- Reverse: Three pointers (prev, curr, next). O(N) time.
- Length: Traverse and count. O(N).

Edge Cases (All Problems):

- Empty list (head=null)
- Single node
- Index=0 or index=size-1
- Invalid index (>=size)

7. Topic Summary (Quick Revision)

Key Points:

- Node = data + next
- Always use temp pointer
- Head for access, never modify directly
- $O(1)$ head ops, $O(N)$ tail/index ops

Patterns:

1. Pointer reversal/swap
2. Two-pass traverse (find prev, then modify)
3. Dummy node for edge cases (advanced)
4. Slow-fast pointers (cycles, middle)

Tricks:

1. DRAW pointers FIRST!
2. Test: null, 1-node, 2-node cases
3. Mention complexities with/without tail
4. "Temp saves head" - key interview phrase

Topic 1 COMPLETE!

Next up: Doubly Linked List. Reply "continue" or any question, and I'll proceed with full depth!

next topic

#####

TOPIC: Doubly Linked List

#####

1. Simple & Detailed Theory (Teacher Style)

Now imagine the **train cars** have **hooks on BOTH ends** - front AND back! Each car knows its previous neighbor AND next neighbor. This is a **Doubly Linked List** - **bidirectional navigation!**

Like a **two-way street** vs singly's one-way street. You can go forward (next) OR backward (prev)!

Why Doubly over Singly?

- Reverse traversal $O(N)$ instead of $O(N)$ reverse
- Delete node if you have node reference (no need to find prev)
- Browser tabs (next/prev tab)

- Undo/Redo stacks

Step-by-step intuition:

1. Node: data + prev + next
2. Head.prev = null, Tail.next = null
3. Insert: Update 4 pointers (new.prev, new.next, prev.next, next.prev)
4. Delete: Update 4 pointers around the node
5. Display forward/backward: Traverse next/prev

Real-life: Playlist with "Next" and "Previous" buttons!

Trade-off: More memory (extra prev pointer), slightly complex code.

2. Core Concepts (Deep Understanding)

Node Structure:

```
class Node {
    int data;
    Node next;
    Node prev;
    Node(int data) {
        this.data = data;
        this.next = null;
        this.prev = null;
    }
}
```

Boundary Conditions:

- Head: prev = null
- Tail: next = null
- Insert between A ↔ B: A.next=B, prev=newNode, newNode.prev=A, newNode.next=B

Insertion Logic (General):

```
To insert NEW between PREV and CURR:
1. NEW.prev = PREV
2. NEW.next = CURR
3. PREV.next = NEW
4. CURR.prev = NEW
```

Time Complexities: (With head/tail)

Operation	Time	Why Better than Singly?
Insert Head	O(1)	Same
Insert Tail	O(1)	Same

Operation	Time	Why Better than Singly?
Delete (given node)	O(1)	O(1) vs O(N) find prev!
Reverse Traverse	O(N)	O(N) vs O(N) reverse
Insert/Delete Index	O(N)	Same

Memory: $O(N)$ nodes $\times$ 3 pointers = $3N$ space.

Key Insight: Symmetric operations - forward/backward mirror each other!

3. Examples (Very Important)

Example 1: Insert at Head

```
Initial: null ← 10 ↔ 20 → null
Insert 5: 5 ↔ 10 ↔ 20
Steps: 5.next=10, 10.prev=5, head=5
```

Example 2: Delete Node 10

```
List: null ← 5 ↔ 10 ↔ 20 → null
5.next=20, 20.prev=5 → null ← 5 ↔ 20 → null
```

Example 3: Insert After Node 10 (value 15)

```
Find 10, curr=10, next=20
15.prev=10, 15.next=20, 10.next=15, 20.prev=15
→ 10 ↔ 15 ↔ 20
```

4. Important Interview Notes ⚠

Frequently Asked:

1. "Implement Doubly LL operations"
2. "Convert Singly to Doubly"
3. "Delete node in O(1) given node reference"
4. "LRU Cache" (uses Doubly LL)
5. "Palindrome check" (reverse traversal)

Tricky Observations:

- Maintain BOTH directions - forget one → broken links!
- Head/Tail updates on boundary inserts/deletes
- Given node delete: `prev.next = node.next; node.next.prev = node.prev`

Common Confusions:

- ✗ Insert: Update only next pointers (breaks backward!)
- ✓ Update ALL 4 pointers

- ✗ Delete head: Forget head = head.next
- ✓ Update boundaries

- ✗ Display reverse from head → start from tail!
- ✓ Traverse prev from tail

Pro Tip: Say "4 pointer updates" - shows you understand doubly complexity!

5. Problem Identification

From video summary (Doubly Linked List Operations):

1. Insertion at first position
2. Insertion at last position
3. Insertion after specific node
4. Display forward
5. Display reverse

Additional standard problems:

6. Deletion of specific node (O(1) if node given)
7. Find length

6. Solve Each Problem (VERY IMPORTANT)

☒ Problem 1: Insertion at First Position

Problem Statement

- Input: Head, data
- Output: New head
- Constraints: $N \leq 10^5$

Intuition: New head.prev=null, oldHead.prev=newHead.

Pattern Used: Boundary Pointer Update

Code (Java):

```
public Node insertFirst(Node head, int data) {
    Node newNode = new Node(data);
    newNode.next = head;
    if (head != null) {
        head.prev = newNode;
    }
    newNode.prev = null;
}
```

```
    return newNode; // New head!  
}
```

Dry Run: Insert 5 in 10↔20

```
newNode.next=10  
10.prev=5  
5.prev=null  
→ null←5↔10↔20 ✓
```

Edge: Empty list → newNode alone.

☒ Problem 2: Insertion at Last Position

Intuition: Traverse to tail, attach.

Code:

```
public Node insertLast(Node head, int data) {  
    Node newNode = new Node(data);  
    if (head == null) return newNode;  
  
    Node temp = head;  
    while (temp.next != null) {  
        temp = temp.next;  
    }  
    temp.next = newNode;  
    newNode.prev = temp;  
    newNode.next = null;  
    return head;  
}
```

Dry Run: 10↔20 insert 30

```
temp=20 (next=null)  
20.next=30, 30.prev=20  
→ 10↔20↔30→null
```

☒ Problem 3: Insertion After Specific Node

Problem Statement

- Input: Head, target value, new data
- Output: Updated head
- Find node with target value, insert after it.

Intuition: Locate target, treat as "prev", insert between target & target.next.

Code:

```

public Node insertAfter(Node head, int targetVal, int newData) {
    Node temp = head;
    while (temp != null && temp.data != targetVal) {
        temp = temp.next;
    }
    if (temp == null) return head; // Target not found

    Node newNode = new Node(newData);
    newNode.next = temp.next;
    newNode.prev = temp;
    temp.next = newNode;

    if (newNode.next != null) {
        newNode.next.prev = newNode;
    }
    return head;
}

```

Detailed Dry Run ★ (List: null←10↔20↔30→null, insertAfter(20, 25)):

```

1. temp=10 (≠20) → temp=20 ✓
2. newNode(25).next = 20.next=30
3. newNode.prev = 20
4. 20.next = 25
5. 30.prev = 25 (since newNode.next=30 != null)
Result: 10↔20↔25↔30

```

Visual pointer changes:

Before: 20.next=30, 30.prev=20

After: 20.next=25, 25.next=30, 25.prev=20, 30.prev=25

Edge Cases: Target not found (no change), insert after tail (new tail), single node.

☒ Problem 4: Display Forward

Code:

```

public void displayForward(Node head) {
    Node temp = head;
    while (temp != null) {
        System.out.print(temp.data + " ↔ ");
        temp = temp.next;
    }
    System.out.println("null");
}

```

☒ Problem 5: Display Reverse

Intuition: Start from tail, go prev.

Code:

```
public void displayReverse(Node tail) { // Need tail!
    Node temp = tail;
    while (temp != null) {
        System.out.print(temp.data + " ↔ ");
        temp = temp.prev;
    }
    System.out.println("null");
}
```

Note: Interviews often ask to find tail first if only head given.

Additional: O(1) Delete Given Node

```
public void deleteNode(Node node) {
    if (node.next == null) { // Last node
        node.prev.next = null;
        return;
    }
    node.prev.next = node.next;
    node.next.prev = node.prev;
}
```

Magic: No traversal needed!

Edge Cases (All): Empty, single node, head/tail ops, invalid target.

7. Topic Summary (Quick Revision)

Key Points:

- **Node:** data + prev + next
- **4 pointer updates** per insert/delete
- **Bidirectional** = reverse easy, delete O(1) with node ref
- **Boundaries:** head.prev=null, tail.next=null

Patterns:

1. Symmetric forward/backward ops
2. O(1) delete with node pointer
3. LRU Cache backbone
4. Two-way traversal

Tricks:

1. "Doubly needs 4 updates" - impress interviewer!
2. Test bidirectional after each op
3. Mention LRU use-case
4. Draw BOTH arrows always!
5. Tail for reverse display

Topic 2 COMPLETE!

Next: Circular Linked List. Say "next topic" to continue! Any questions on Doubly?

next

```
#####  
TOPIC: Circular Linked List  
#####
```

1. Simple & Detailed Theory (Teacher Style)

Take the train analogy further: Now the last car hooks back to the engine! It's a continuous loop - like a racetrack or roundtable. No beginning or end - circular!

In a Circular Linked List (CLL):

- Singly CLL: Tail.next = Head (not null)
- Doubly CLL: Tail.next=Head AND Head.prev=Tail

Why Circular?

- Infinite loop simulation (round-robin scheduling)
- Efficient insert/delete at any position (if you have pointer)
- Queue implementations (no null checks)
- Music playlist looping

Step-by-step intuition:

1. Node same as singly/doubly
2. Single entry point: Usually head OR tail
3. Traverse: Stop when back to start (temp == head)
4. Insert: Break link, insert new, reconnect circle
5. Delete: Bypass node, reconnect neighbors to circle

Real-life: Clock hands (12 connects to 1), carousel ride!

Key Difference: NO NULL - always cycle back!

2. Core Concepts (Deep Understanding)

Singly Circular Node:

```
Last Node: next = HEAD (not null!)  
Traverse condition: while(temp != head) OR count-based
```

Boundary Rules:

- Empty list: Single node, node.next = itself
- Insert after node: Similar to singly, but handle if inserting after tail
- Delete: If deleting head → find new head (second node)

Insertion Logic:

```
To insert NEW after REF in circle:  
1. NEW.next = REF.next  
2. REF.next = NEW  
3. If REF was tail: Update tail
```

Display Logic:

```
temp = head;  
do {  
    print(temp.data);  
    temp = temp.next;  
} while(temp != head);
```

Time Complexities: (Head-only, no tail)

Operation	Singly CLL	vs Singly Linear
Insert Head	O(1)	Same
Insert End	O(N)	O(N)
Delete (given ptr)	O(1)	O(N)
Traverse	O(N)	Same
Cycle detection	Built-in!	Extra work

Memory: Same as linear + circular link.

Pro Insight: Tail reference makes end ops O(1), but head-only simpler for interviews.

3. Examples (Very Important)

Example 1: Basic Circle

```
Single node 10: 10→10 (self-loop)
Two nodes: 10→20→10
Three: 10→20→30→10
```

Example 2: Insert After 20 (in 10→20→30→10)

```
20.next was 30 → NEW(25).next=30
20.next=25 → 10→20→25→30→10
```

Example 3: Delete 20

```
Prev(10).next = 20.next(25)
→ 10→25→30→10
```

4. Important Interview Notes [△](#)

Frequently Asked:

1. "Implement Circular LL operations"
2. "Detect cycle in linked list" (CLL always has!)
3. "Josephus problem" (circular elimination)
4. "Round Robin scheduling simulation"
5. "Convert linear to circular"

Tricky Observations:

- Self-loop for single node
- Traverse stop: `temp != head` (not null!)
- Delete head: Second node becomes new head
- Empty list handling

Common Confusions:

- ✗ `while(temp.next != null) → INFINITE LOOP!`
✓ `while(temp != head)`
- ✗ Forget `tail.next=head` after insert
✓ Always close the circle
- ✗ Display without do-while → miss last node

Pro Tip: "Circular eliminates null checks but adds cycle management"

5. Problem Identification

From video summary (Circular Linked List Operations):

1. Insertion into circular structure
2. Display/Traversing circular list
3. Deletion of specific value

Additional standard problems:

4. Insert at position
5. Josephus Problem (count every k-th)

6. Solve Each Problem (VERY IMPORTANT)

☒ Problem 1: Insertion (After Specific Value)

Problem Statement

- Input: Head, target value, new data
- Output: Updated head
- Constraints: $N \leq 10^4$, singly circular

Intuition: Find target, insert after it like singly, circle auto-maintained.

Pattern Used: Circular Traversal + Pointer Swap

Code (Java):

```
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}

public Node insertAfterValue(Node head, int targetVal, int newData) {
    if (head == null) return new Node(newData); // Empty → new single node circle

    Node newNode = new Node(newData);
    Node temp = head;

    // Traverse circle
    do {
        if (temp.data == targetVal) {
            newNode.next = temp.next;
            temp.next = newNode;
            return head;
        }
        temp = temp.next;
    } while (temp != head);
}
```

```
    return head; // Target not found
}
```

Detailed Dry Run ★ (Head=10→20→30→10, insertAfter(20,25)):

```
1. temp=10 (≠20), temp=20 ✓
2. newNode.next = 20.next=30
3. 20.next = newNode(25)
4. Loop stops (don't continue traversal)
Result: 10→20→25→30→10 ✓
```

Visual:

Before: 20→30

After: 20→25→30 (circle intact)

Variables:

- temp starts head(10)
- Finds 20 at step 2
- newNode.next=30
- 20.next=25 ✓

Edge: Target=head (insert after head), single node (self-loop maintained).

☒ Problem 2: Display Circular List

Problem Statement

- Input: Head
- Output: Print all nodes

Intuition: Do-while to handle circle.

Code:

```
public void displayCircular(Node head) {
    if (head == null) return;

    Node temp = head;
    do {
        System.out.print(temp.data + " → ");
        temp = temp.next;
    } while (temp != head);
    System.out.println("(back to head)");
}
```

Dry Run: 10→20→30→10

```
temp=10 print10 →20
print20 →30
print30 →10 STOP!
```

Mistake: Forgetting do-while → infinite or miss nodes.

☒ Problem 3: Deletion of Specific Value

Problem Statement

- Input: Head, value to delete
- Output: Updated head (may change if delete head)

Intuition: Find prev of target, prev.next = target.next.

Code:

```
public Node deleteValue(Node head, int val) {
    if (head == null) return null;

    // Single node case
    if (head.data == val && head.next == head) {
        return null;
    }

    Node temp = head;
    Node prev = null;

    // Find target and prev
    do {
        if (temp.data == val) {
            if (prev == null) { // Deleting head
                // Find new head (second node)
                Node newHead = temp.next;
                Node tail = head;
                while (tail.next != head) {
                    tail = tail.next;
                }
                tail.next = newHead;
                return newHead;
            } else {
                prev.next = temp.next;
                return head;
            }
        }
        prev = temp;
        temp = temp.next;
    } while (temp != head);

    return head; // Not found
}
```

Detailed Dry Run ★ (Delete 20 from 10→20→30→10):

1. temp=10 (≠20), prev=10, temp=20 ✓
 2. prev=20? No, prev=10 from loop
 3. prev.next(10.next) = 20.next=30
 4. Circle: 10→30→10 (20 bypassed)
- Head unchanged=10

Special: Delete 10 (head):

1. temp=10==10, prev=null → Delete head mode
2. newHead=10.next=20
3. Find tail: tail=10→30 (30.next=10==head), tail=30
4. 30.next=20
5. Return 20 (new head) →20→30→20 ✓

Edge Cases:

- Delete only node → null
- Delete head → update tail.next to second
- Delete last → prev.next=head
- Not found → no change
- Infinite loop risk → do-while prevents

Additional: Insert at Position Similar to singly but circular traverse.

7. Topic Summary (Quick Revision)

Key Points:

- Tail.next = Head (singly), Head.prev=Tail (doubly)
- Traverse: do-while(temp != head)
- No null → special empty/single cases
- Head delete → find new head + update tail

Patterns:

1. Circular traversal (do-while)
2. Josephus (k-th elimination)
3. Round-robin
4. Cycle detection baseline

Tricks:

1. "Do-while prevents infinite loop!"
2. Always verify circle closes (tail.next==head)
3. Test delete head - trickiest!
4. Mention OS scheduling use-case
5. Draw circle diagram

ALL TOPICS COMPLETE! ☑

You've now mastered **Linked Lists** from this video:

- ✓ Singly (linear one-way)
- ✓ Doubly (bidirectional)

- ✓ Circular (looping)

Quick Check: Can you code insert-after in doubly CLL?

Next Steps:

- Practice LeetCode: 206(Reverse), 141(Cycle), 2(Add Two Numbers)
- Ask for problems/quiz on linked lists
- New video summary?

What would you like next? More DSA topics, problems, or new summary? ☒

STEP 1: Topic Identification

Here are all the main topics extracted from the video summary:

1. Cycle Detection in Linked Lists (Cycle detection, length, start + Happy Number)
2. List Manipulation Basics (Recursive insertion, remove duplicates, merge sorted lists)
3. Finding Middle & Sorting
4. Reversal Techniques (Full reverse recursive/iterative, Reverse LinkedList II, k-node reverses)
5. Advanced Reordering (Palindrome, Reorder List, Rotate List)

I'll teach these ONE AT A TIME starting with the most critical FAANG pattern.

```
#####
TOPIC: Cycle Detection in Linked Lists
#####
```

1. Simple & Detailed Theory (Teacher Style)

Imagine you're walking on a circular track but **don't know it's circular**. You keep going and **suddenly recognize a landmark you saw before** - you're in a cycle!

In linked lists, a cycle exists when a node's **next** eventually loops back to a previous node (tail connects to some node in middle).

Real-life analogy: Browser with broken "Back" button creating infinite history loop!

Why cycles matter in interviews?

- Memory leaks detection (production debugging)
- Graph algorithms (cycle detection = graph cycle)
- FAANG favorite - tests two-pointer genius
- Foundation for Happy Numbers, linked list intersection

Step-by-step intuition:

1. Naive: Use Set $\rightarrow O(N)$ space

2. Smart: Two runners - slow (1 step), fast (2 steps)

3. Collision: If fast laps slow → cycle exists!

4. Math magic: Distance analysis finds cycle start

Key Insight: In circular track, fast turtle laps slow tortoise at 2:1 ratio!

2. Core Concepts (Deep Understanding)

Floyd's Tortoise-Hare (Cycle Detection):

```
slow = fast = head
while(fast && fast.next) {
    slow = slow.next
    fast = fast.next.next
    if(slow == fast) → CYCLE!
}
```

Why it works (Math):

Let cycle start at distance = D from head
Cycle length = C
When slow enters cycle, fast has headstart = D steps
Relative speed = 1 step/loop
They meet when fast gains D steps on slow → first meeting mod C

Cycle Properties:

- Meeting point distance from cycle start: $(D \% C)$
- Cycle length: Distance slow travels from meeting → back to meeting
- Cycle start: Reset slow to head, move both 1-step till collision

Time/Space:

Method	Time	Space
HashSet	$O(N)$	$O(N)$
Two Pointer	$O(N)$	$O(1)$
Brute Force	$O(N^2)$	$O(1)$

FAANG Expectation: Always mention $O(1)$ space two-pointer!

3. Examples (Very Important)

Example 1: Cycle Detection

```
List: 1→2→3→4→5→3 (cycle starts at 3)
Step 1: slow=1, fast=1
Step 2: slow=2, fast=3
```



```
Step 3: slow=3, fast=5  
Step 4: slow=4, fast=3 → MEET! Cycle ✓
```

Example 2: No Cycle

```
1→2→3→null  
fast reaches null → No cycle
```

Example 3: Cycle Length

```
Meet at 5: 5→3→4→5 (length=3)
```

4. Important Interview Notes ⚠

FAANG Frequency: Top 5 LinkedList question!

```
Google: "Detect cycle O(1) space"  
Amazon: "Find cycle start"  
Facebook: "Cycle length"  
Microsoft: "Multiple cycles"
```

Tricky Questions:

1. "Prove time complexity $O(N)$ " → Draw distance diagram!
2. "Single node cycle?" → Yes, self-loop
3. "Head in cycle?" → Works perfectly
4. "Multiple entry points?" → Still detects

Common Mistakes:

```
✗ fast.next.next without null check → NPE!  
✓ if(!fast || !fast.next) return false;  
  
✗ slow==fast only once → false negatives  
✓ Loop until collision or null
```

5. Problem Identification

From video:

1. LinkedList Cycle (detect existence)
2. Length of LinkedList Cycle
3. LinkedList Cycle II (find start)
4. Happy Number (cycle in number sequence)

LeetCode References:

- 141. Linked List Cycle
- 142. Linked List Cycle II
- 202. Happy Number

6. Solve Each Problem (VERY IMPORTANT)

📌 Problem 1: LinkedList Cycle (LeetCode 141)

Problem Statement

Input: head of singly linked list
 Output: true if cycle exists, false otherwise
 Constraints: $0 \leq \text{node count} \leq 5000$, cycle possible

Intuition (Interview Approach):

1. "Two runners - tortoise(1x), hare(2x)"
2. "If hare laps tortoise $\rightarrow$ cycle"
3. "Handle null carefully"
4. Time: $O(N)$, Space: $O(1)$

Pattern Used: Fast-Slow Pointers

Brute Force: HashSet track visited $\rightarrow O(N)$ space

Optimized Approach:

1. Initialize slow=fast=head
2. Move slow 1x, fast 2x
3. Collision $\rightarrow$ cycle ✓

Code (Java):

```
public boolean hasCycle(Node head) {
    if (head == null || head.next == null) return false;

    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

Detailed Dry Run ★

Input: 1→2→3→4→2 (cycle at index 1)

Iter0: slow=1, fast=1

Iter1: slow=2, fast=3

Iter2: slow=3, fast=2 (3.next.next=2.next=3)

Iter3: slow=4, fast=3

Iter4: slow=2, fast=2 → RETURN true!

Visual:

```

      ----
      |    |
      V    |
1→2→3→4→2
s→ → → → f (meet at 2)
```

☒ Problem 2: Length of LinkedList Cycle

Problem Statement: Return cycle length after detecting cycle

Approach: After meeting point, move slow until back to meeting.

Code:

```
public int cycleLength(Node head) {
    Node slow = head, fast = head;

    // Phase 1: Detect cycle
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) break;
    }

    if (fast == null || fast.next == null) return 0;

    // Phase 2: Count cycle length
    int length = 1;
    slow = slow.next; // Move past meeting point
    while (slow != fast) {
        slow = slow.next;
        length++;
    }
    return length;
}
```

Dry Run: Meet at 2, cycle: 2→3→4→2 (length=3)

☒ Problem 3: LinkedList Cycle II (LeetCode 142)

Problem Statement: Return node where cycle begins

Math Magic:

Meeting point distance from cycle start = (distance to cycle) % cycleLength
Reset slow=head, move both 1x → meet at cycle start!

Code:

```
public Node detectCycle(Node head) {
    Node slow = head, fast = head;

    // Phase 1: Find meeting point
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) break;
    }

    if (fast == null || fast.next == null) return null;

    // Phase 2: Move slow to head
    slow = head;

    // Phase 3: Move both 1 step
    while (slow != fast) {
        slow = slow.next;
        fast = fast.next;
    }
    return slow; // Cycle start!
}
```

Detailed Dry Run ★ (1→2→3→4→2):

Phase 1: Meet at 2 (as above)
Phase 2: slow=1, fast=2
Phase 3:
 Iter1: slow=2, fast=3
 Iter2: slow=3, fast=4
 Iter3: slow=4, fast=2 → MEET AT 2 (cycle start!)

☒ Problem 4: Happy Number (LeetCode 202)

Problem Statement:

A happy number is where repeated sum of squared digits → 1
Unhappy → enters cycle ≠ 1
Use cycle detection on NUMBER sequence!

Code:

```
public boolean isHappy(int n) {
    Set<Integer> seen = new HashSet<>();
    while (n != 1 && !seen.contains(n)) {
        seen.add(n);
        n = getSum(n);
    }
    return n == 1;
}

private int getSum(int n) {
    int sum = 0;
    while (n > 0) {
        int d = n % 10;
        sum += d * d;
        n /= 10;
    }
    return sum;
}
```

Cycle insight: Unhappy numbers cycle: 4→16→37→58→89→145→42→20→4

Edge Cases:

1. head=null → false
2. Single node → check self-loop
3. Cycle at head → return head
4. Cycle length=2 → still works
5. Fast reaches null → no cycle

7. Topic Summary (Quick Revision)

Key Points:

- Two pointers: slow(1x), fast(2x)
- Collision = cycle (O(N), O(1) space)
- Cycle start: Reset + same speed
- Cycle length: Post-collision loop count

Patterns:

1. Fast-Slow (90% LL problems)
2. Cycle detection → Happy Numbers
3. Distance math (meeting point analysis)
4. Floyd's algorithm (CS classic)

Interview Tricks:

1. "Two runners analogy" → interviewer loves!
2. Draw diagram: head→cycle→meeting
3. Always mention O(1) space
4. Practice: 141, 142, 202 LeetCode
5. Edge: null, single node self-loop

Topic 1 COMPLETE!

Next: List Manipulation Basics (insert, duplicates, merge). Say "next topic" to continue!

Quick Test: Can you explain why fast-slow meets at $(\text{distance} \% \text{cycleLength})$ from cycle start? ☒



next

```
#####  
TOPIC: List Manipulation Basics  
#####
```

1. Simple & Detailed Theory (Teacher Style)

Think of linked lists like a conveyor belt of packages. Sometimes you need to add packages smartly (recursive insertion), remove identical packages (duplicates), or combine two belts (merge sorted lists).

These are FAANG bread-and-butter operations - every interviewer expects you to nail them!

Why these matter:

- Recursive insertion = recursion mastery + base cases
- Remove duplicates = in-place modification pattern
- Merge sorted = divide & conquer + pointer juggling

Real-life analogy:

- Warehouse sorter removing duplicate SKUs
- Merging two sorted inventory lists
- Recursively unpacking nested boxes

Step-by-step intuition:

1. Recursion: "Smaller problem + combine result"
2. Duplicates: "Current same as next? Skip it!"
3. Merge: "Pick smaller head, advance that pointer"

Key Insight: All use traversal + pointer manipulation - master these, conquer 70% LL problems!

2. Core Concepts (Deep Understanding)

Recursive Traversal Logic:

```
insert(head) =  
  if head.next == null: create new node  
  else: head.next = insert(head.next)  
  return head
```

In-place Duplicate Removal:

```
Pattern: while(curr && curr.next && curr.val == curr.next.val)  
  curr.next = curr.next.next
```

Merge Two Sorted Lists:

```
Dummy node trick:  
dummy → smaller head  
while both lists have nodes:  
  attach smaller, advance that pointer  
attach remaining list
```

Time/Space Complexities:

Operation	Time	Space	In-place?
Recursive Insert	$O(N)$	$O(N)$ recursion	Yes
Remove Duplicates	$O(N)$	$O(1)$	Yes
Merge Two Lists	$O(N+M)$	$O(1)$	Yes

FAANG Expectation: Always prefer $O(1)$ space when possible!

3. Examples (Very Important)

Example 1: Remove Duplicates

```
Input: 1→1→2→3→3  
Process:  
  1==1? Skip →1→2→3→3  
  3==3? Skip →1→2→3  
Output: 1→2→3
```

Example 2: Merge Sorted

```
L1: 1→3→5    L2: 2→4→6  
Step1: 1<2 →1→(3vs2)  
Step2: 2<3 →1→2→(3vs4)  
Step3: 3<4 →1→2→3→(5vs4)
```

```
...  
Output: 1→2→3→4→5→6
```

Example 3: Recursive Insert (at end)

```
insert(1→2→3, 4):  
    3.next=null →3→4  
    2.next=3→4  
    1.next=2→3→4  
Output: 1→2→3→4
```

4. Important Interview Notes [⚠](#)

Most Asked Variations:

```
Google: "Merge K sorted lists" (heap)  
Amazon: "Remove duplicates from unsorted"  
Facebook: "Recursive vs iterative tradeoffs"  
Microsoft: "In-place vs new list"
```

Tricky Points:

1. **Recursive insert**: Base case + return head!
2. **Duplicates**: Single pass, modify pointers only
3. **Merge**: Dummy node prevents head handling

Common Mistakes:

- ✗ Recursion: Forget "return head"
- ✗ Duplicates: curr.next.next null check missing
- ✗ Merge: Wrong pointer advance → data loss!

Pro Tip: Always draw dummy node for merges!

5. Problem Identification

From video:

1. Recursive Insertion in LinkedList
2. Remove Duplicates from Sorted LinkedList (LeetCode 82)
3. Merge Two Sorted LinkedLists (LeetCode 21)

6. Solve Each Problem (VERY IMPORTANT)

☒ Problem 1: Recursive Insertion (Insert at End)

Problem Statement

Input: head, value
Output: head with value inserted at end (recursive)
Constraints: $N \leq 10^4$

Intuition:

"Help my next node insert, then I'll point to its result"
Base: last node → create new
Recursive: head.next = insert(head.next)

Pattern Used: Recursion + Tail Insertion

Code (Java):

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}

public ListNode insertEndRecursive(ListNode head, int val) {
    // Base case: empty or last node
    if (head == null || head.next == null) {
        ListNode newNode = new ListNode(val);
        if (head != null) {
            head.next = newNode;
            return head;
        }
        return newNode;
    }

    // Recursive case
    insertEndRecursive(head.next, val);
    return head;
}
```

Detailed Dry Run ★

Input: 1→2→null, insert 3

Call1: head=1, head.next=2 → recurse(2,3)
Call2: head=2, head.next=null → 2.next=3 ✓ RETURN 2
Call1: 1.next remains 2 ✓ RETURN 1

Stack unwind:

2→3
1→2→3 ✓

📌 Problem 2: Remove Duplicates from Sorted LinkedList (LeetCode 82)

Problem Statement

Input: head of sorted LL with possible duplicates
Output: head with duplicates removed
Must use O(1) space, modify in-place
Example: 1→1→2→3→3 → 1→2→3

Intuition:

Single pass: When `curr.val == curr.next.val`, skip next
Continue while duplicates exist at current position

Pattern Used: In-place Pointer Jump

Code:

```
public ListNode deleteDuplicates(ListNode head) {  
    ListNode curr = head;  
  
    while (curr != null && curr.next != null) {  
        if (curr.val == curr.next.val) {  
            // Skip duplicate  
            curr.next = curr.next.next;  
        } else {  
            // Move to next unique  
            curr = curr.next;  
        }  
    }  
    return head;  
}
```

Detailed Dry Run ★

Input: 1→1→2→3→3→null

Iter1: curr=1, 1==1 ✓ → 1.next=2 (skip second 1)
Iter2: curr=1, 1!=2 → curr=2
Iter3: curr=2, 2!=3 → curr=3
Iter4: curr=3, 3==3 ✓ → 3.next=null (skip last 3)
Iter5: curr=3, curr.next=null → STOP

Result: 1→2→3→null ✓

Edge Cases:

All duplicates: 1→1→1 → null ✓

No duplicates: 1→2→3 ✓

Single node ✓

📌 Problem 3: Merge Two Sorted LinkedLists (LeetCode 21)

Problem Statement

Input: head1, head2 (both sorted)

Output: Single sorted merged list

Constraints: Total nodes $\leq 10^4$

Intuition:

1. Dummy node simplifies head handling
2. Compare heads, take smaller, advance that pointer
3. Attach remaining list at end

Pattern Used: Dummy Node + Two Pointer Merge

Code:

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;

    while (l1 != null && l2 != null) {
        if (l1.val < l2.val) {
            tail.next = l1;
            l1 = l1.next;
        } else {
            tail.next = l2;
            l2 = l2.next;
        }
        tail = tail.next;
    }

    // Attach remaining
    tail.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}
```

Detailed Dry Run ★

l1: 1→3→5 l2: 2→4→6 dummy→?

Iter1: 1<2 → dummy→1, l1=3, tail=1

Iter2: 3>2 → 1→2, l2=4, tail=2

Iter3: 3<4 → 2→3, l1=5, tail=3

Iter4: 5>4 → 3→4, l2=6, tail=4

```
Iter5: 5 < 6 → 4 → 5, l1=null, tail=5
Iter6: l1 null → 5 → 6, done

Final: dummy → 1 → 2 → 3 → 4 → 5 → 6 ✓
```

Edge Cases:

```
One empty: l1=null → return l2 ✓
Both empty → null ✓
Equal values ✓
```

7. Topic Summary (Quick Revision)

Key Points:

- **Recursion:** Base case + "help my neighbor"
- **Duplicates:** `while (curr.val == curr.next.val) skip`
- **Merge:** Dummy + compare heads + advance smaller

Patterns:

1. Recursion (stack → tail insertion)
2. In-place modification (pointer jump)
3. Dummy node (head simplification)
4. Two pointer merge (sorted arrays same!)

Interview Tricks:

1. "Dummy node prevents 6 cases → 1 case"
2. Draw merge: l1, l2 heads → tail grows
3. Always test: all dups, one empty, equal vals
4. Mention: "Same as merge sort merge step"
5. Time $O(N)$, Space $O(1)$ every time!

Topic 2 COMPLETE!

Next: Finding Middle & Sorting. Say "next topic" to continue!

Quick Check: Why does dummy node make merge easier? (Head handling!) ☒

✱✱

next

```
#####
TOPIC: Finding Middle & Sorting
#####
```

1. Simple & Detailed Theory (Teacher Style)

Finding Middle: Imagine a race between two runners on a linked list track. One runs slowly (1 step), the other doubly fast (2 steps). When the fast runner finishes the race (reaches end), the slow runner is exactly at the middle!

Sorting: Linked lists can't use array tricks (index swap). Instead, use Merge Sort - divide → conquer → merge. Perfect for linked lists because merge is $O(N)$ and we get stable $O(N \log N)$!

Real-life analogy:

- Middle: Two friends walking - fast friend reaches end, slow friend halfway
- Sort: Sorting exam papers by splitting stacks, sorting halves, merging sorted halves

Why FAANG loves these:

- Middle: Tests two-pointer mastery (cycle detection cousin!)
- Sort: Tests divide & conquer + merge pattern (system design too!)

Step-by-step intuition:

1. Middle: `while (fast && fast.next) { slow=slow.next; fast=fast.next.next }`
2. Sort: Find middle → sort(left) → sort(right) → merge(sorted halves)

2. Core Concepts (Deep Understanding)

Middle Finding (Two Pointer):

```
Distance covered when fast finishes:
Fast: 2N steps = full list
Slow: N steps = exactly middle!

Even length: slow at 2nd middle (standard)
Odd length: slow at exact middle ✓
```

Merge Sort for Linked Lists:

1. Find middle (two pointer)
2. Split: `left=1→mid`, `right=mid.next→end`
3. Recurse: `sort(left)`, `sort(right)`
4. Merge: `sorted_left + sorted_right`

Time Complexities:

Operation	Time	Space	Method
Find Middle	$O(N)$	$O(1)$	Two Pointer
Count + Traverse	$O(N)$	$O(1)$	Two pass
Merge Sort	$O(N \log N)$	$O(\log N)$ recursion	Optimal

Operation	Time	Space	Method
Bubble Sort	$O(N^2)$	$O(1)$	Too slow!

Key Insight: Same two-pointer for cycles AND middle!

3. Examples (Very Important)

Example 1: Find Middle

```
Odd: 1→2→3→4→5
    s→ → →f→
    slow=3 (middle)

Even: 1→2→3→4
    s→ →f
    slow=3 (2nd middle) ✓
```

Example 2: Merge Sort Split

```
Input: 4→2→1→3
Middle=1 (index 2)
Split: 4→2 | 1→3
Sort:   2→4 | 1→3
Merge: 1→2→3→4
```

4. Important Interview Notes ⚠

Frequently Asked:

```
Google: "Find middle  $O(1)$  space"
Amazon: "Sort linked list  $O(N \log N)$ "
Facebook: "Why merge sort, not quicksort?"
Microsoft: "Even vs odd length middle?"
```

Tricky Variations:

1. "Return BOTH middles" (even case)
2. "Delete middle node"
3. "Insert at middle"
4. "Merge sort with duplicates"

Common Confusions:

- ✗ Two pass counting → "Inefficient!"
- ✓ Single pass two-pointer

✗ Bubble sort → "Too slow $O(N^2)!$ "

✓ Merge sort $O(N \log N)$

5. Problem Identification

From video:

1. Middle of the LinkedList (LeetCode 876)
2. Sort LinkedList (LeetCode 148)

6. Solve Each Problem (VERY IMPORTANT)

📌 Problem 1: Middle of the LinkedList (LeetCode 876)

Problem Statement

Input: head of singly linked list
Output: middle node (2nd middle for even length)
Constraints: $1 \leq \text{nodes} \leq 100$, no cycle

Intuition (Interview Approach):

1. "Two runners analogy - fast laps track twice!"
2. "When fast finishes, slow = middle"
3. Single pass, $O(1)$ space!

Pattern Used: Fast-Slow Pointers

Brute Force: Count length → traverse length/2 → $O(N)$ time, $O(1)$ space

Optimized: Two pointers single pass.

Code (Java):

```
public ListNode middleNode(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

Detailed Dry Run ★

```

**Odd length: 1→2→3→4→5**
Iter0: s=1, f=1
Iter1: s=2, f=3
Iter2: s=3, f=5
Iter3: f.next=null → STOP, return 3 ✓

**Even length: 1→2→3→4**
Iter0: s=1, f=1
Iter1: s=2, f=3
Iter2: f.next=null → STOP, return 2 ✓ (2nd middle)

```

📌 Problem 2: Sort LinkedList (LeetCode 148)

Problem Statement

```

Input: head of singly linked list (unsorted)
Output: sorted linked list ascending
Constraints: -5000 <= val <= 5000, nodes <= 5k
Expect: O(N log N) time, O(1) space

```

Intuition:

1. "Can't random access → Merge Sort perfect!"
2. "Find middle → split → sort both → merge"
3. Recursion tree: $\log N$ levels $\times$ N merge = $N \log N$

Pattern Used: Merge Sort + Two Pointer Split

Code (Complete Solution):

```

public ListNode sortList(ListNode head) {
    // Base case
    if (head == null || head.next == null) return head;

    // Step 1: Find middle
    ListNode mid = getMiddle(head);
    ListNode nextOfMid = mid.next;

    // Step 2: Split
    mid.next = null;

    // Step 3: Recurse
    ListNode left = sortList(head);
    ListNode right = sortList(nextOfMid);

    // Step 4: Merge
    return merge(left, right);
}

private ListNode getMiddle(ListNode head) {
    ListNode slow = head;

```



```

    ListNode fast = head.next; // Start fast ahead!

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}

private ListNode merge(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;

    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) {
            tail.next = l1;
            l1 = l1.next;
        } else {
            tail.next = l2;
            l2 = l2.next;
        }
        tail = tail.next;
    }

    tail.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}

```

Detailed Dry Run ★ (4→2→1→3):

```

**Level 1:** head=4→2→1→3
mid=1 (slow=4→2→1, fast=null)
Split: 4→2 | 1→3

**Level 2 Left:** 4→2
mid=2 → 4 | 2

**Level 3:** 4 (base), 2 (base)

**Merge up:**
2→4 + 1→3 = 1→2→3→4 ✓

Visual tree:
      [4→2→1→3]
     /       \
   [4→2]     [1→3]
  /  \     /  \
[ ^9_4] [ ^9_2] [ ^9_1] [ ^9_3]

```

Edge Cases:

```

Empty: null ✓
Single: return ✓

```

Two nodes: swap if needed ✓
All equal values ✓

7. Topic Summary (Quick Revision)

Key Points:

- Middle: `slow(1x)+fast(2x)` → single pass $O(N)$
- Sort: Merge Sort = split + recurse + merge
- Even length → 2nd middle (standard)

Patterns:

1. Two pointers (middle, split, cycles)
2. Merge pattern (sort, merge lists)
3. Divide & conquer ($\log N$ recursion)
4. Dummy node (head handling)

Interview Tricks:

1. "Same two-pointer as cycle detection!"
2. Draw recursion tree for merge sort
3. Always say " $O(N \log N)$ optimal for comparison sort"
4. Test even/odd lengths
5. Mention: "Stable sort" ✓

Topic 3 COMPLETE!

Next: Reversal Techniques (recursive, iterative, k-nodes). Say "next topic" to continue!

Quick Check: Why does fast start at `head.next` in `getMiddle()`? ☒

✱✱

next

#####

TOPIC: Reversal Techniques

#####

1. Simple & Detailed Theory (Teacher Style)

Reversing a linked list is like rewinding a tape - each link that pointed forward now points backward. Imagine turning around every train car so the engine becomes the caboose!

Three levels of mastery:

1. Iterative: 3 pointers dancing (prev, curr, next)

2. Recursive: "Help my neighbor reverse, then I'll follow"

3. K-groups: Reverse chunks like shuffling cards

Real-life analogy:

- Full reverse = reading book backward
- K-groups = reversing every paragraph separately
- Alternate K = reverse odd paragraphs only

Why FAANG loves reversals?

- Tests pointer manipulation fluency
- Building block for palindromes, reordering
- Recursive vs iterative tradeoff discussion
- Edge cases (k=1, k>n, empty list)

Step-by-step intuition:

```
Iterative: prev←curr→next becomes next←curr→prev
Recursive: reverse(head.next) then head.next.prev = head
K-groups: Find k nodes → reverse group → repeat
```

2. Core Concepts (Deep Understanding)

Iterative Reversal (3-Pointer Dance):

```
prev = null
while(curr) {
    next = curr.next;    // Save lost link!
    curr.next = prev;    // Reverse!
    prev = curr;         // Progress
    curr = next;         // Progress
}
head = prev;            // New head!
```

Recursive Reversal:

```
reverse(head) =
    if head.next==null: return head (new head)
    newHead = reverse(head.next)
    head.next.next = head    // Reverse link
    head.next = null        // Cut forward
    return newHead
```

K-Group Reversal:

1. Count k nodes
2. If <k nodes: return original

3. Reverse k-group
4. Recurse on remaining
5. Connect: prev_group_tail → reversed_head → remaining

Time/Space:

Method	Time	Space
Iterative Full	O(N)	O(1)
Recursive Full	O(N)	O(N) stack
K-Group	O(N)	O(1) iterative

3. Examples (Very Important)

Example 1: Iterative Full Reverse

```

1→2→3→null
↓
P→1→2→3
  ↓next=2
  1→P (null)
  P=1, C=2
↓
1←2→3
  ↓next=3
  2→1
  P=2, C=3
↓
1←2←3→null ✓

```

Example 2: Recursive

```

reverse(1→2→3):
  reverse(2→3):
    reverse(3): return 3
    3.next=2, 2.next=null →2→3
    return 2
  2.next.next=1, 1.next=null →2→3, 1 cut
  return 2 ✓

```

Example 3: K=2 Groups

```

1→2→3→4→5   k=2
↓
2→1→3→4→5
  ↓
  4→3→5 ✓

```

4. Important Interview Notes ⚠

FAANG Patterns:

```
Google: "Reverse recursively + iteratively"  
Amazon: "Reverse k-groups" (hard)  
Facebook: "Reverse alternate k-nodes"  
Microsoft: "In-place reversal proof"
```

Tricky Questions:

1. "k=1?" → No change
2. "Last group <k?" → Don't reverse
3. "k > n?" → Don't reverse
4. "Prove no memory leak"

Common Mistakes:

- ✗ Forget `next = curr.next` → DATA LOSS!
- ✗ Recursive: `head.next.next = head` (order!)
- ✗ K-groups: Forget connect `prev→newHead`

5. Problem Identification

From video:

1. Reverse LinkedList (Recursive)
2. Reverse LinkedList (Iterative)
3. Reverse LinkedList II (positions m to n)
4. Reverse k-Nodes
5. Reverse Alternate k-Nodes

LeetCode: 206, 92, 25

6. Solve Each Problem (VERY IMPORTANT)

📌 Problem 1: Reverse LinkedList (Iterative) - LeetCode 206

Problem Statement

```
Input: head of singly linked list  
Output: reversed list head  
Constraints: 1 <= N <= 5000
```

Intuition: "3 pointers prevent losing the chain!"

Pattern Used: Triple Pointer Rotation

Code:

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;

    while (curr != null) {
        ListNode next = curr.next; // SAVE!
        curr.next = prev;          // REVERSE
        prev = curr;                // PROGRESS
        curr = next;               // PROGRESS
    }
    return prev; // New head!
}
```

Detailed Dry Run ★

```
Input: 1→2→3→null

**Iter1:** P=null, C=1, next=2
           1→null, P=1, C=2
**Iter2:** P=1, C=2, next=3
           2→1,   P=2, C=3
**Iter3:** P=2, C=3, next=null
           3→2,   P=3, C=null ✓
Return P=3 → 3→2→1 ✓
```

☒ Problem 2: Reverse LinkedList (Recursive)

Code:

```
public ListNode reverseListRecursive(ListNode head) {
    // Base case
    if (head == null || head.next == null) {
        return head;
    }

    // Recurse
    ListNode newHead = reverseListRecursive(head.next);

    // Reverse current link
    head.next.next = head;
    head.next = null;

    return newHead;
}
```

Dry Run: reverse(1→2→3) → 3→2→1 (stack unwind reverses links)

☒ Problem 3: Reverse LinkedList II (m to n) - LeetCode 92

Problem Statement

Reverse nodes from position m to n (1-indexed)
1→2→3→4→5→NULL, m=2,n=4 →1→4→3→2→5

Intuition:

1. Reach m-1 (prev segment)
2. Reverse m→n segment
3. Connect: prev_seg → reversed → rest

Code:

```
public ListNode reverseBetween(ListNode head, int m, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode prev = dummy;

    // Step 1: Reach m-1
    for (int i = 1; i < m; i++) {
        prev = prev.next;
    }

    // Step 2: Reverse m to n
    ListNode start = prev.next;      // m-th node
    ListNode then = start.next;      // (m+1)-th

    for (int i = m; i < n; i++) {
        start.next = then.next;
        then.next = prev.next;
        prev.next = then;
        then = start.next;
    }

    return dummy.next;
}
```

Dry Run (m=2,n=4): 1→[4→3→2]→5 ✓

☒ Problem 4: Reverse k-Nodes (LeetCode 25)

Problem Statement

Reverse every k nodes, last group <k unchanged
1→2→3→4→5→6 k=2 →2→1→4→3→6

Code (Simplified):

```

public ListNode reverseKGroup(ListNode head, int k) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode prevGroupTail = dummy;

    while (true) {
        // Check if k nodes exist
        ListNode tail = prevGroupTail;
        for (int i = 0; i < k; i++) {
            tail = tail.next;
            if (tail == null) return dummy.next;
        }

        // Reverse k nodes
        ListNode groupHead = prevGroupTail.next;
        reverseGroup(prevGroupTail, tail);
        prevGroupTail = groupHead;
    }
}

private void reverseGroup(ListNode prevGroupTail, ListNode groupTail) {
    ListNode prev = prevGroupTail;
    ListNode curr = prev.next;

    while (curr != groupTail.next) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    // Connect groups
    prevGroupTail.next = groupTail;
}

```

7. Topic Summary (Quick Revision)

Key Points:

- **Iterative:** $prev \leftarrow curr \rightarrow next \rightarrow next \leftarrow curr \rightarrow prev$
- **Recursive:** $reverse(next); next.next = head; head.next = null$
- **K-groups:** Count $\rightarrow$ reverse $\rightarrow$ connect $\rightarrow$ repeat

Patterns:

1. Triple pointer (iterative)
2. Recursion stack unwind
3. Group reversal (k-parameterized)
4. Dummy node (boundary cases)

Interview Tricks:

1. "3 pointers prevent data loss!"
2. Draw iterative dance step-by-step
3. Recursive: "Base → unwind reverses"
4. K-groups: Always check "k nodes left?"
5. Test: k=1 (identity), k>n (no change)

Topic 4 COMPLETE!

Next (Final): Advanced Reordering (Palindrome, Reorder, Rotate). Say "next topic"! ☒



next

#####

TOPIC: Advanced Reordering

#####

1. Simple & Detailed Theory (Teacher Style)

Advanced reordering takes basic reversal to genius level - think palindromes (mirror check), reordering (1st+last+2nd+2ndLast), and rotation (circular shift).

Real-life analogies:

- Palindrome: "Racecar" reads same forward/backward
- Reorder: Shuffle cards: top+bottom+2nd+2ndBottom
- Rotate: Ferris wheel - shift everyone k positions

Why FAANG loves these HARD problems:

- Multi-technique combo (reverse+middle+merge)
- In-place mastery (no extra space)
- Pattern recognition (reuse cycle/middle/reverse)
- Debugging complex pointers

Master technique: All combine middle finding + reversal + merging!

Step-by-step intuition:

1. Palindrome: Find middle → reverse 2nd half → compare
2. Reorder: Middle → reverse 2nd → interleave merge
3. Rotate: Length N → k%N → split+swap heads

2. Core Concepts (Deep Understanding)

Palindrome Algorithm:

1. Find middle (two pointer)
2. Reverse 2nd half
3. Compare 1st half vs reversed 2nd
4. Restore original (reverse back) OR modify in-place

Reorder List Pattern:

```
1→2→3→4→5 → 1→5→2→4→3  
Find middle(3) → 1→2 | 3→4→5  
Reverse 2nd: 1→2 | 5→4→3  
Interleave: 1→5→2→4→3
```

Rotate Math:

```
Length=5, k=2 → rotate right: 4→5→1→2→3  
Equivalent: tail.next=head, newHead = len-k+1 node
```

Time/Space: All $O(N)$ time, $O(1)$ space (FAANG holy grail!)

3. Examples (Very Important)

Palindrome: 1→2→2→1

```
Middle=2(first), reverse 2→1 →1→2  
Compare: 1→2 vs 1→2 ✓ PALINDROME
```

Reorder: 1→2→3→4→5

```
Middle=3, rev2nd=5→4→3  
Interleave: 1→5→2→4→3 ✓
```

Rotate k=2: 1→2→3→4→5

```
Len=5, k=2 → newHead=4th node (4)  
4→5→1→2→3 ✓
```

4. Important Interview Notes [△](#)

FAANG Hard Questions:

```
Google: "Palindrome  $O(1)$  space"  
Amazon: "Reorder List in-place"
```

Facebook: "Rotate without array"
Microsoft: "All 3 in 1 interview!"

Tricky Points:

1. **Even vs odd palindrome**: middle handling
2. **Reorder**: Stop interleave at shorter length
3. **Rotate**: $k \% \text{length}$ (avoid TLE!)

Common Mistakes:

- ✗ **Palindrome**: Compare without reversing 2nd half
- ✗ **Reorder**: Wrong interleave pointers
- ✗ **Rotate**: Forget $k \% \text{len}$ or circular connection

5. Problem Identification

From video:

1. **Palindrome LinkedList** (LeetCode 234)
2. **Reorder LinkedList** (LeetCode 143)
3. **Rotate LinkedList** (LeetCode 61)

6. Solve Each Problem (VERY IMPORTANT)

☒ Problem 1: Palindrome LinkedList (LeetCode 234)

Problem Statement

Input: head of singly linked list
Output: true if palindrome, false otherwise
Constraints: $N \leq 10^5$, $O(1)$ space preferred
Ex: $1 \rightarrow 2 \rightarrow 2 \rightarrow 1 \rightarrow \text{true}$

Intuition:

"Mirror check: reverse 2nd half, compare with 1st"
Single pass middle + reverse + compare = $O(N)$, $O(1)$

Pattern Used: Middle + Reverse + Compare

Code (Java):

```
public boolean isPalindrome(ListNode head) {  
    // Step 1: Find middle  
    ListNode slow = head, fast = head;
```

```

    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    // Step 2: Reverse 2nd half
    ListNode secondHalf = reverse(slow.next);

    // Step 3: Compare halves
    return compareLists(head, secondHalf);
}

private ListNode reverse(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

private boolean compareLists(ListNode left, ListNode right) {
    while (left != null && right != null) {
        if (left.val != right.val) return false;
        left = left.next;
        right = right.next;
    }
    return true;
}

```

Detailed Dry Run ☆ (1→2→2→1):

```

**Step1 Middle:** slow=2(first), fast=end
**Step2 Reverse:** 2→1 becomes 1→2
**Step3 Compare:** 1→2 vs 1→2 ✓ true

**Odd case 1→2→3→2→1:**
Middle=3, rev 2→1 →1→2
Compare 1→2→3 vs 1→2 ✓ (stop at shorter)

```

☒ Problem 2: Reorder LinkedList (LeetCode 143)

Problem Statement

```

Reorder: 1→2→3→4→5 → 1→5→2→4→3
In-place modification, no return value

```

Intuition:

"Find middle → reverse 2nd → interleave merge"
Same as palindrome first 2 steps + zipper merge!

Code:

```
public void reorderList(ListNode head) {
    if (head == null || head.next == null) return;

    // Step 1: Find middle
    ListNode slow = head, fast = head.next;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    // Step 2: Split + reverse 2nd half
    ListNode second = reverse(slow.next);
    slow.next = null; // Split!

    // Step 3: Interleave merge
    ListNode first = head;
    while (second != null) {
        ListNode next1 = first.next;
        ListNode next2 = second.next;

        first.next = second;
        second.next = next1;

        first = next1;
        second = next2;
    }
}
```

Detailed Dry Run ☆ (1→2→3→4→5):

```
**Middle:** slow=3
**Split+Rev:** 1→2 | 5→4→3
**Interleave:**
Iter1: 1→5→2 | 4→3
Iter2: 1→5→2→4→3 ✓
```

📌 Problem 3: Rotate LinkedList (LeetCode 61)

Problem Statement

Rotate right k places: 1→2→3→4→5 k=2 →4→5→1→2→3
Return new head

Intuition:

1. Make circular: `tail.next = head`
2. `k %= length` (optimization)
3. New head = (len-k)th node
4. Break circle at kth node

Code:

```
public ListNode rotateRight(ListNode head, int k) {
    if (head == null || head.next == null) return head;

    // Step 1: Find length + tail
    int len = 1;
    ListNode tail = head;
    while (tail.next != null) {
        tail = tail.next;
        len++;
    }

    // Step 2: k %= len, connect tail->head
    k %= len;
    if (k == 0) return head;
    tail.next = head;

    // Step 3: Find (len-k)th node = new head
    ListNode newTail = head;
    for (int i = 0; i < len - k - 1; i++) {
        newTail = newTail.next;
    }

    ListNode newHead = newTail.next;
    newTail.next = null;

    return newHead;
}
```

Detailed Dry Run ☆ (1→2→3→4→5, k=2):

```
len=5, k=2%5=2
tail=5.next=1 (circular!)
newTail: start1→2→3 (len-k-1=2 steps)
newHead=4
3.next=null
→4→5→1→2→3 ✓
```

7. Topic Summary (Quick Revision)

Key Points:

- Palindrome: Middle→reverse→compare
- Reorder: Middle→reverse→interleave

- Rotate: $\text{len} + k \% \text{len} \rightarrow \text{circular} \rightarrow \text{split}$

Master Patterns:

1. Two-pointer middle (ALL problems!)
2. Reverse 2nd half (palindrome+reorder)
3. In-place merge/interleave
4. Circular list manipulation

Interview Gold:

1. "All use middle+reverse pattern!"
2. Draw: $\text{original} \rightarrow \text{middle} \rightarrow \text{reversed} \rightarrow \text{final}$
3. Always $O(N)$ time, $O(1)$ space
4. Test: even/odd lengths, $k=0, k > \text{len}$
5. LeetCode: 234, 143, 61

ALL 5 TOPICS COMPLETE! ☑

You've conquered Linked List mastery:

1. ✓ Cycle Detection (fast-slow)
2. ✓ Manipulation (insert+merge+duplicates)
3. ✓ Middle+Sort (two-pointer+mergeSort)
4. ✓ Reversals (iterative+recursive+k-groups)
5. ✓ Advanced (palindrome+reorder+rotate)

FAANG Ready! Can solve 95% LinkedList problems.

Next Steps:

- Practice LeetCode Top 20 LL problems
- Mock interview: Explain any solution
- New DSA topic/video summary?

Quick Test: Why `fast.next.next` in palindrome middle finding? ☑



1. <https://www.youtube.com/watch?v=2PQDsQA1j3o>
2. <https://www.youtube.com/watch?v=hwpNzen0z0w>
3. <https://dev.to/somadevtoo/i-failed-4-faang-interviews-before-learning-this-the-complete-coding-interview-preparation-guide-gjh>
4. <https://www.youtube.com/watch?v=bFWMIG5jvQI>
5. <https://www.youtube.com/watch?v=xP3x4K4Rvi4>
6. https://www.linkedin.com/posts/kunalkr19_dsa-roadmap-for-beginners-activity-7381334485759528960-aCzq

7. <https://www.youtube.com/watch?v=bfEHKYP0lyU>
8. https://www.linkedin.com/posts/vivekbalyan0503_dsa-faang-google-activity-7439115171148709888-9NTY
9. <https://www.youtube.com/watch?v=gMcPQmrplQY>
10. <https://www.designgurus.io/answers/detail/which-is-the-best-youtube-channel-to-learn-dsa>
11. https://www.linkedin.com/posts/akashsinnggh_6-linked-list-patterns-you-must-know-for-activity-7344210579164651521-R28j
12. <https://www.geeksforgeeks.org/dsa/top-50-linked-list-interview-question/>
13. <https://www.youtube.com/watch?v=SIKgDtpJnxg>
14. <https://algotcademy.com/blog/mastering-linked-list-problems-for-interviews-a-comprehensive-guide/>
15. <https://www.techinterviewhandbook.org/algorithms/linked-list/>
16. <https://www.scribd.com/document/783918529/linkedlist-leetcode>
17. <https://in.indeed.com/career-advice/interviewing/linked-list-interview-questions>
18. <https://www.youtube.com/watch?v=70tx7KcMROc>
19. <https://leetcode.com/discuss/interview-question/1376426/Google-or-Facebook-or-Interview-questions-or-July-2020-July-2021/>
20. <https://www.youtube.com/watch?v=Laugw5Uj57g>
21. <https://www.youtube.com/watch?v=70tx7KcMROc>
22. <https://blog.devgenius.io/mastering-linked-lists-ddba473e1c41>
23. https://www.linkedin.com/posts/akashsinnggh_6-linked-list-patterns-you-must-know-for-activity-7344210579164651521-R28j
24. <https://www.youtube.com/watch?v=SIKgDtpJnxg>
25. <https://in.indeed.com/career-advice/interviewing/linked-list-interview-questions>
26. <https://www.instagram.com/p/DSSOjRoEYQL/>
27. <https://algotcademy.com/blog/mastering-linked-list-problems-for-interviews-a-comprehensive-guide/>
28. <https://www.interviewbit.com/linked-list-interview-questions/>
29. <https://www.techinterviewhandbook.org/algorithms/linked-list/>
30. <https://leetcode.com/discuss/interview-question/1376426/Google-or-Facebook-or-Interview-questions-or-July-2020-July-2021/>
31. https://www.youtube.com/watch?v=8ocB7a_c-Cc
32. <https://www.enjoyalgorithms.com/blog/find-middle-of-linked-list/>
33. <https://takeuforward.org/linked-list/sort-a-linked-list>
34. <https://www.youtube.com/watch?v=7LjQ57RqgEc>
35. <https://leetcode.com/problems/sort-list/>
36. https://www.reddit.com/r/leetcode/comments/1b8t3pf/these_are_my_two_approaches_to_find_middle_of_a/
37. <https://youcademy.org/reverse-linked-list-iterative-recursive-approach/>

38. <https://www.geeksforgeeks.org/dsa/recursive-reversing-a-linked-list-a-simple-implementation/>
39. <https://algo.monster/liteproblems/25>
40. <https://leetcode.com/problems/reverse-nodes-in-k-group/>
41. <https://blog.devgenius.io/reverse-a-linked-list-iterative-and-recursive-methods-c-e2399eb8f085>
42. <https://www.geeksforgeeks.org/dsa/reverse-alternate-k-nodes-in-a-singly-linked-list/>
43. <https://www.interviewbit.com/problems/reverse-alternate-k-nodes/>
44. <https://www.geeksforgeeks.org/dsa/reverse-a-linked-list/>
45. <https://www.geeksforgeeks.org/dsa/reverse-a-linked-list-in-groups-of-given-size/>
46. <https://takeuforward.org/data-structure/reverse-linked-list-in-groups-of-size-k>
47. <https://www.geeksforgeeks.org/dsa/function-to-check-if-a-singly-linked-list-is-palindrome/>
48. <https://takeuforward.org/data-structure/check-if-given-linked-list-is-plindrome>
49. <https://algotmap.io/question-bank/reorder-list>
50. <https://www.geeksforgeeks.org/dsa/rotate-a-linked-list/>
51. <https://leetcode.com/problems/palindrome-linked-list/>
52. [https://github.com/doocs/leetcode/blob/main/solution/0100-0199/0143.Reorder List/README_EN.md](https://github.com/doocs/leetcode/blob/main/solution/0100-0199/0143.Reorder%20List/README_EN.md)
53. <https://leetcode.com/problems/rotate-list/>
54. https://www.youtube.com/watch?v=IRY_G-u_8jk
55. <https://www.youtube.com/watch?v=Pno7rUOZM-o>
56. <https://www.geeksforgeeks.org/problems/rotate-a-linked-list/1>